



Génie du Logiciel et des Systèmes

Rapport de Mini-Projet

Chaîne de vérification de modèles de processus

Groupe B-13

Magron Mathis

Nancey Mathis

Comparetto Matthieu

Département Sciences du Numérique

Deuxième année

2025 — 2026

Table des matières

1	Introduction	2
2	Métamodèles (avec Ecore)	2
2.1	SimplePDL	2
2.2	PetriNet	2
3	Sémantique statique (avec Java)	3
3.1	Contraintes SimplePDL	3
3.1.1	Contraintes sur le processus	3
3.1.2	Contraintes sur les WorkDefinitions (activités)	3
3.1.3	Contraintes sur les WorkSequences (dépendances)	3
3.1.4	Contraintes sur les Guidances	3
3.1.5	Contraintes sur les Ressources	3
3.1.6	Contraintes sur les Ressources nécessaires	3
3.2	Contraintes PetriNet	4
3.2.1	Contraintes sur le réseau (Petri)	4
3.2.2	Contraintes sur les places	4
3.2.3	Contraintes sur les transitions	4
3.2.4	Contraintes sur les arcs	4
4	Eclipse Modeling Framework (EMF)	4
4.1	Plugin SimplePDL	4
4.2	Plugin PetriNet	5
5	Transformation de modèle à texte (avec Acceléo)	6
5.1	SimplePDL → HTML	6
5.2	SimplePDL → DOT	6
5.3	PetriNet → TINA	7
6	Définition de syntaxes concrètes graphiques (avec Sirius)	7
6.1	SimplePDL	7
7	Définition de syntaxes concrètes textuelles (avec Xtext)	7
7.1	SimplePDL.xtext	7
8	Transformation de modèle à modèle (avec ATL)	7
8.1	SimplePDL → PetriNet	7
9	Validation	7
10	Conclusion et ajouts par rapport à l'oral	8

1 Introduction

Ce mini-projet, réalisé dans le cadre du module d'Ingénierie Dirigée par les Modèles (IDM), a pour objectif la mise en place d'une chaîne de vérification de modèles de processus décrits en *SimplePDL*. L'objectif principal est de vérifier si un processus peut se terminer ou non, en le transformant en un réseau de Petri, puis en utilisant l'outil de model-checking *Tina*.

Nous présentons dans ce rapport les différentes étapes de réalisation, depuis la modélisation des métamodèles jusqu'à la validation de la transformation, en passant par la définition de syntaxes concrètes et la prise en compte des ressources.

2 Métamodèles (avec Ecore)

2.1 SimplePDL

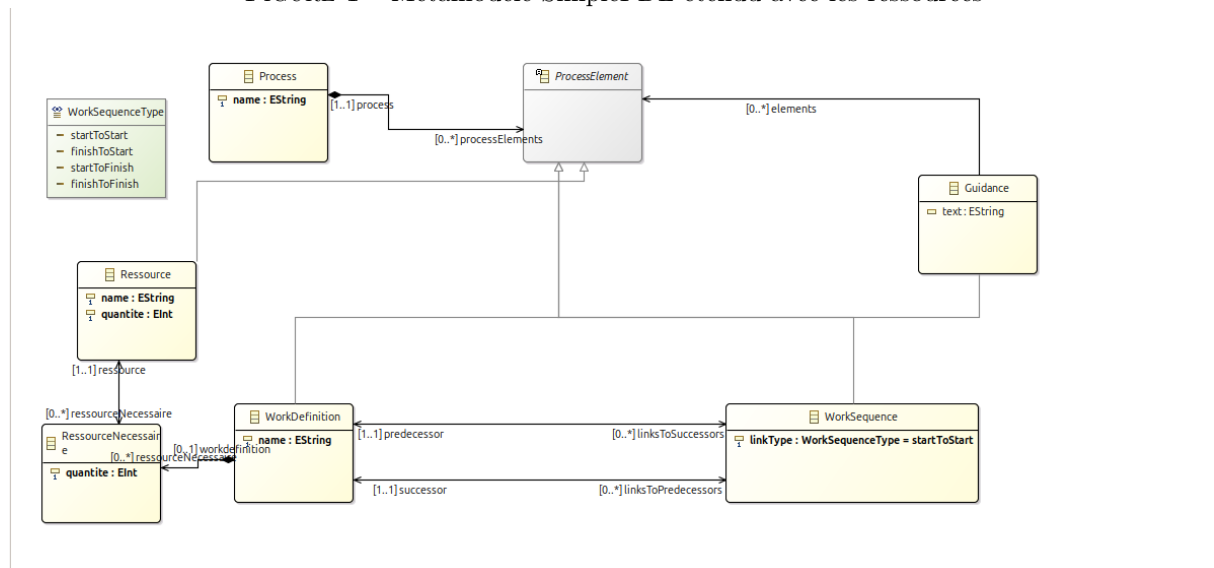
Le métamodèle *SimplePDL* initial a été étendu pour prendre en compte les **ressources** et les **Guidances**, comme demandé dans la tâche T_1 . Nous avons introduit les concepts suivants :

- **Resource** : représente un type de ressource (humaine, outil, etc.) avec une quantité disponible.
- **RessourceNecessaire** : associe une ressource à une *WorkDefinition* et spécifie la quantité requise.
- **Guidance** : permet d'écrire un commentaire.

Les relations entre ces éléments sont bidirectionnelles (**EOpposite**) pour faciliter la navigation dans le modèle.

[FIGURE : Métamodèle SimplePDL étendu avec les ressources]

FIGURE 1 – Métamodèle SimplePDL étendu avec les ressources



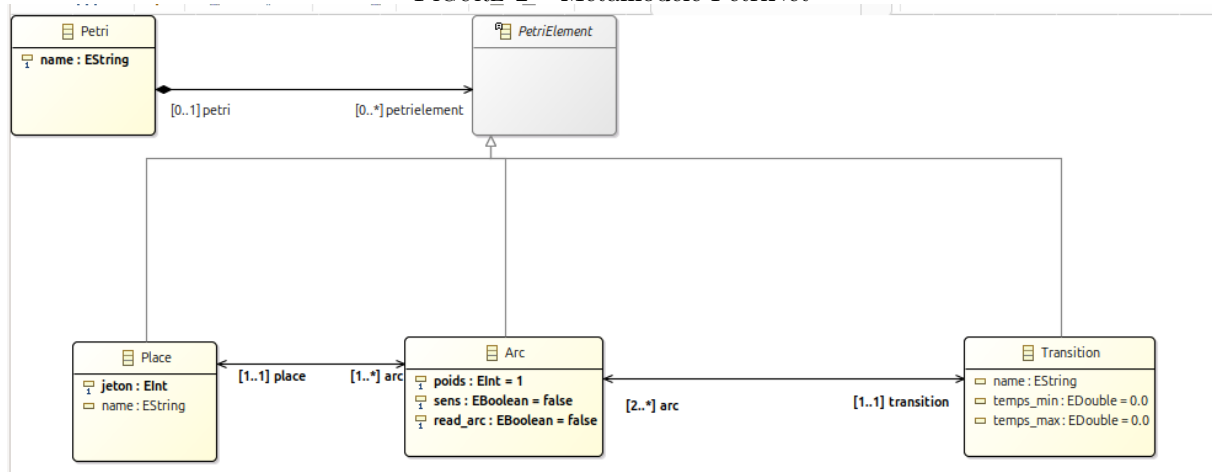
2.2 PetriNet

Le métamodèle *PetriNet* a été conçu pour représenter les réseaux de Petri de manière générique. Il comprend :

- **Petri** : conteneur principal.
- **PetriElement** : classe abstraite pour les places et transitions.
- **Place** : avec un attribut **tokens**.
- **Transition**.
- **Arc** : avec un attribut **quantité**, un booléen **sens**, un booléen **readOnly**, un int **tempsMax** et un int **tempsMin** permettant d'introduire le temps pour les transitions.

[FIGURE : Métamodèle PetriNet]

FIGURE 2 – Métamodèle PetriNet



3 Sémantique statique (avec Java)

Les contraintes de cohérence ont été implémentées en Java (via des streams), comme demandé dans le sujet. Voici le détail des contraintes validées :

3.1 Contraintes SimplePDL

3.1.1 Contraintes sur le processus

- **Nom** : le nom du processus doit respecter les conventions Java pour les identifiants (commencer par une lettre ou un underscore, suivi de lettres, chiffres ou underscores).

3.1.2 Contraintes sur les WorkDefinitions (activités)

- **Nom** : le nom de l'activité doit respecter les conventions Java pour les identifiants.
- **Nom unique** : le nom de chaque activité doit être unique dans le processus (pas de doublons).

3.1.3 Contraintes sur les WorkSequences (dépendances)

- **Pas de boucle** : une dépendance ne peut pas relier une activité à elle-même.
- **WorkSequence unique** : il ne peut pas exister deux dépendances identiques entre les mêmes activités avec le même type de lien.

3.1.4 Contraintes sur les Guidances

- **Texte non vide** : le texte d'une guidance ne doit pas être vide.

3.1.5 Contraintes sur les Ressources

- **Quantité positive** : la quantité disponible d'une ressource doit être strictement positive.

3.1.6 Contraintes sur les Ressources nécessaires

- **Quantité positive** : la quantité nécessaire pour une ressource doit être strictement positive.
- **Quantité suffisante** : la quantité demandée ne peut pas dépasser la quantité disponible de la ressource.
- **Ressource unique** : une activité ne peut pas demander plusieurs fois la même ressource (pas de doublons).

3.2 Contraintes PetriNet

3.2.1 Contraintes sur le réseau (Petri)

- **Nom** : le nom du réseau doit respecter les conventions Java pour les identifiants.

3.2.2 Contraintes sur les places

- **Nom** : le nom de la place doit respecter les conventions Java pour les identifiants.
- **Quantité positive** : le nombre de jetons dans une place doit être positif ou nul (pas de jetons négatifs).

3.2.3 Contraintes sur les transitions

- **Nom** : le nom de la transition doit respecter les conventions Java pour les identifiants.
- **Temps positif** : le temps minimal d'exécution d'une transition doit être positif ou nul.
- **Temps valide** : le temps maximal doit être supérieur ou égal au temps minimal.
- **Arcs entrants/sortants** : chaque transition doit avoir au moins un arc entrant et un arc sortant.

3.2.4 Contraintes sur les arcs

- **Quantité** : le poids d'un arc doit être supérieur ou égal à 1.
- **Arc complet** : un arc doit relier une place et une transition.
- **ReadArc direction** : un read-arc (arc de lecture) doit aller d'une place vers une transition.

Listing 1 – Exemple de contrainte : unicité des noms

```
1 public Boolean caseWorkDefinition(WorkDefinition object) {
2     // Contraintes sur WD
3     this.result.recordIfFailed(
4         object.getName() != null || object.getName().
5             matches(IDENT_REGEX),
6         object,
7         "Le nom de l'activité ne respecte pas les
8             conventions Java");
9
10    this.result.recordIfFailed(
11        object.getProcess().getProcessElements().stream
12            ()
13            .filter(p -> p.eClass().getClassifierID
14                () == SimplepdlPackage.
15                    WORK_DEFINITION)
16            .allMatch(pe -> (pe.equals(object) ||
17                !((WorkDefinition) pe).getName().
18                    contains(object.getName()))),
19        object,
20        "Le nom de l'activité (" + object.getName() +
21            ") n'est pas unique");
22 }
```

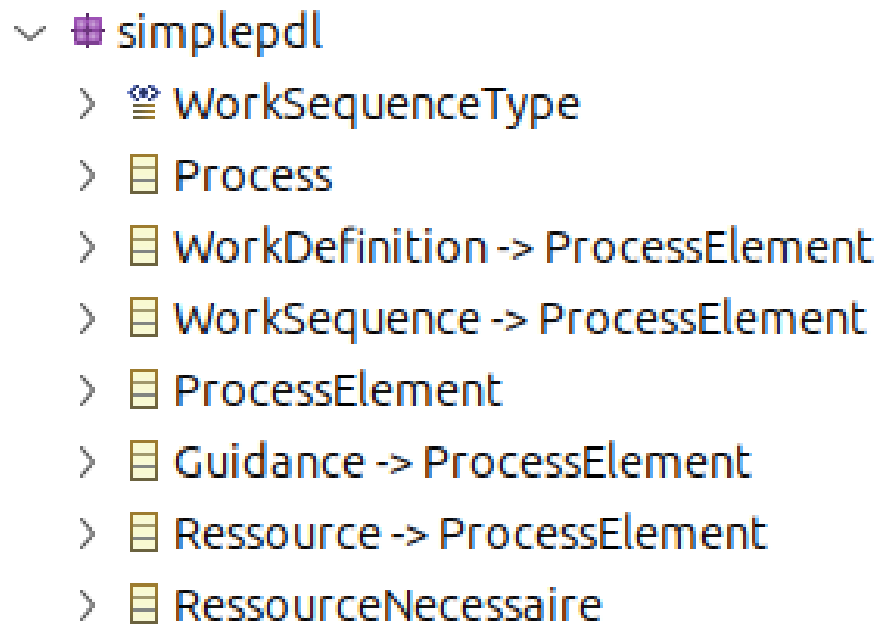
4 Eclipse Modeling Framework (EMF)

4.1 Plugin SimplePDL

Nous avons généré un plugin EMF pour SimplePDL, fournissant un éditeur arborescent pour créer et modifier les modèles SimplePDL.

[FIGURE : Éditeur arborescent d'un modèle SimplePDL]

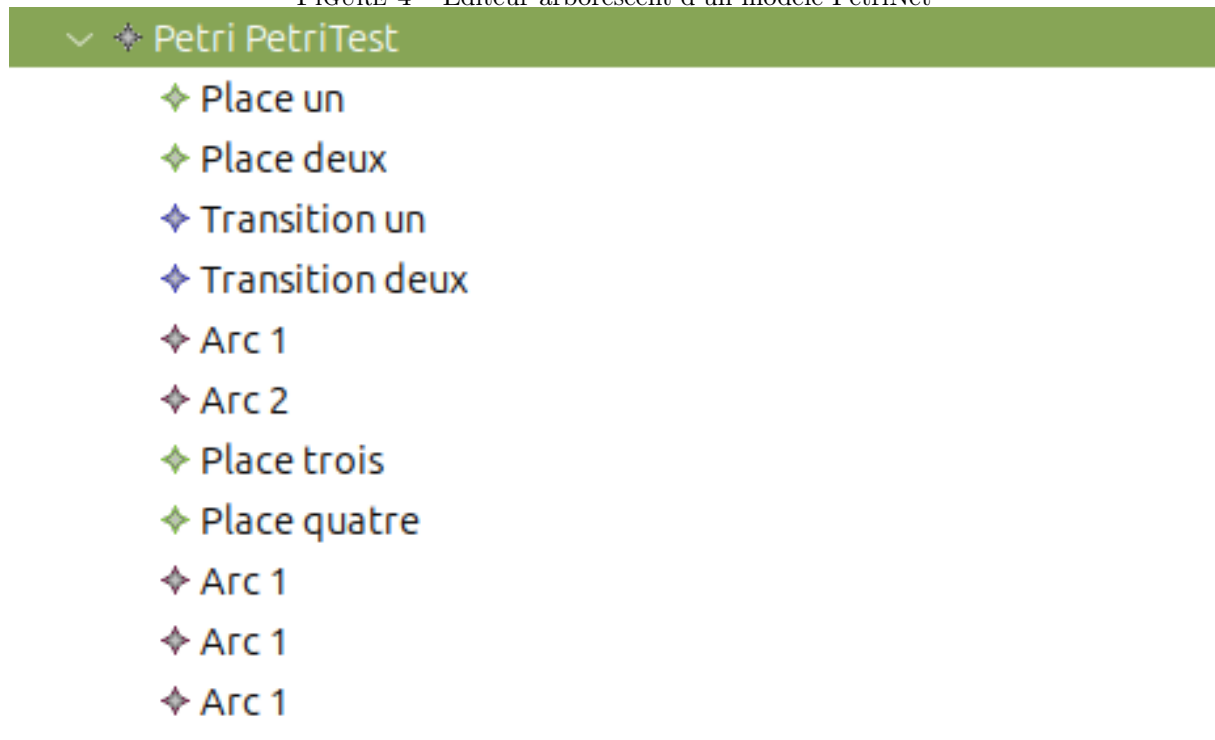
FIGURE 3 – Éditeur arborescent d'un modèle SimplePDL



4.2 Plugin PetriNet

[FIGURE : Éditeur arborescent d'un modèle PetriNet]

FIGURE 4 – Éditeur arborescent d'un modèle PetriNet



De même, un plugin EMF a été créé pour PetriNet, avec son éditeur arborescent dédié.

5 Transformation de modèle à texte (avec Acceléo)

5.1 SimplePDL → HTML

Une transformation Acceléo permet de générer une documentation HTML des processus.

Listing 2 – Exemple de sortie HTML

```
1 <head><title>ExempleProcess1</title></head>
2 <body>
3 <h1>Process "ExempleProcess1"</h1>
4 <h2>Work definitions</h2>
5   <ul>
6     <li>A1</li>
7     <li>A1</li>
8     <li>A2 requires A1 to be started, A2 to be started.</li>
9     <li>A1</li>
10  </ul>
11 </body>
```

5.2 SimplePDL → DOT

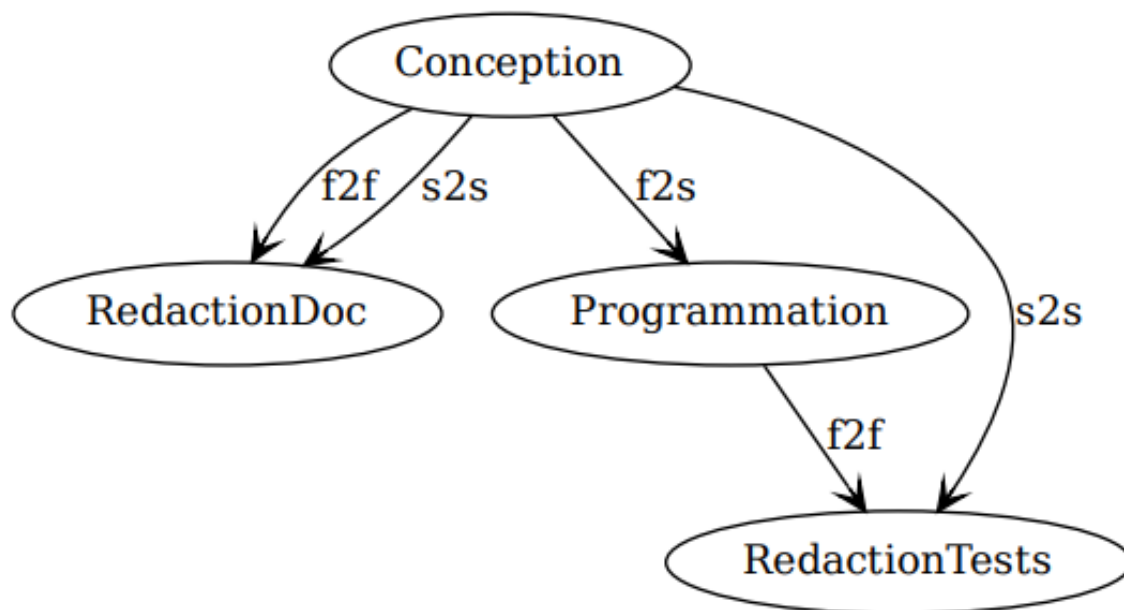
Une transformation vers le format DOT permet de visualiser le graphe du processus.

Listing 3 – Exemple de sortie DOT

```
1 digraph Developpement {
2
3   Conception -> RedactionDoc [arrowhead=vee label=f2f]
4   Conception -> RedactionDoc [arrowhead=vee label=s2s]
5   Conception -> Programmation [arrowhead=vee label=f2s]
6   Conception -> RedactionTests [arrowhead=vee label=s2s]
7   Programmation -> RedactionTests [arrowhead=vee label=f2f]
8 }
```

[FIGURE : Graphe du processus]

FIGURE 5 – Graphe du processus



5.3 PetriNet $\rightarrow$ TINA

La transformation vers le format `.net` de Tina est essentielle pour le model-checking via LTL.

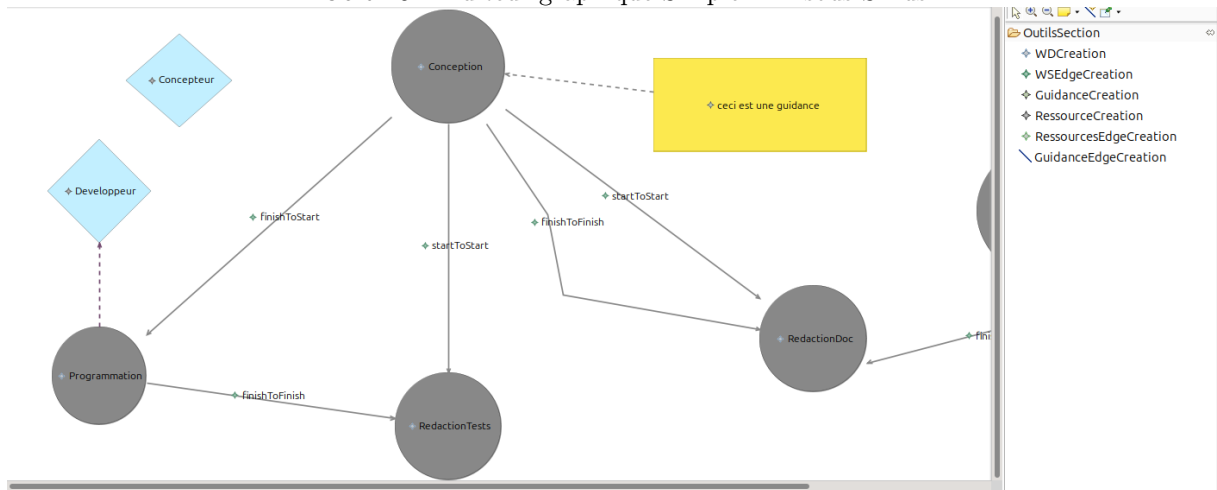
6 Définition de syntaxes concrètes graphiques (avec Sirius)

6.1 SimplePDL

Un éditeur graphique a été défini pour SimplePDL sous Sirius, permettant de visualiser et d'éditer les processus de manière diagrammatique.

[FIGURE : Éditeur graphique SimplePDL sous Sirius]

FIGURE 6 – Éditeur graphique SimplePDL sous Sirius



7 Définition de syntaxes concrètes textuelles (avec Xtext)

7.1 SimplePDL.xtext

Une syntaxe textuelle pour SimplePDL a été définie avec Xtext, permettant de saisir les modèles sous forme textuelle. Nous avons pour cela repris `pd11` du TP, que nous avons complété avec les mots-clés **rs** (pour ressource), **rsn** (pour ressourceNécessaire) ainsi que **quantité**.

Listing 4 – Exemple de modèle SimplePDL avec ressources

```

1 process Developpement {
2   rs Concepteur quantite 3
3   rs Developpeur quantite 2
4   wd Conception rsn { Concepteur quantite 2 }
5   wd Programmation rsn { Developpeur quantite 2 }
6   ws f2s from Conception to Programmation
7 }

```

8 Transformation de modèle à modèle (avec ATL)

8.1 SimplePDL $\rightarrow$ PetriNet

Nous avons réalisé une transformation de SimplePDL vers PetriNet avec ATL.

9 Validation

Des propriétés LTL ont été générées pour vérifier : - la terminaison du processus ; - les invariants. Ces propriétés sont vérifiées avec Tina après transformation, validant ainsi notre chaîne de vérification.

Listing 5 – Propriétés LTL

```

1  op finished = Conception_finished /\ RedactionTests_finished /\
    RedactionDoc_finished /\ Programmation_finished;
2
3  [] (finished => dead);
4  [] <> dead ;
5  [] (dead => finished);
6  - <> finished;
7
8  [] (Conception_ready + Conception_running + Conception_finished = 1);
9  [] (RedactionDoc_ready + RedactionDoc_running + RedactionDoc_finished = 1);
10 [] (Programmation_ready + Programmation_running + Programmation_finished = 1);
11 [] (RedactionTests_ready + RedactionTests_running + RedactionTests_finished = 1)
    ;

```

10 Conclusion et ajouts par rapport à l’oral

Ce mini-projet nous a permis de maîtriser l’ensemble de la chaîne de vérification IDM, depuis la modélisation jusqu’au model-checking. Les extensions (ressources, validation) ont renforcé la robustesse de la transformation. Les différentes approches (Java, ATL, Acceléo) nous ont montré les avantages et limitations de chaque technologie.

Depuis l’oral, nous avons corrigé le fichier `ToTINA.mtl`, qui permet de transformer les modèles PetriNet en Tina. De plus, nous avons corrigé notre transformation ATL pour transformer une `WorkDefinition` en quatre places et deux transitions.